

MLOps Assignment 01 — Heart Disease Risk Prediction

MLOps Assignment 01 — End-to-End Heart Disease Risk API

1. Project overview
2. Setup & install instructions
3. EDA findings
4. Modelling choices
5. Experiment tracking summary (MLflow)
6. Reproducibility
7. CI/CD workflow
8. Containerisation
9. Kubernetes deployment (Docker Desktop)
10. Monitoring & logging
11. Architecture diagram
12. Deliverables checklist
13. Academic integrity statement

MLOps Assignment 01 — End-to-End Heart Disease Risk API

Course: AIMLCZG523 · Machine Learning Operations **Author:** Kanna TJ (2024ad05219@wilp.bits-pilani.ac.in) **Repository:** https://github.com/kannatj/MLOPS_HeartDiseaseRiskPrediction **Deployed API:** Local only — Docker Compose (:8000) or local Kubernetes; run/access instructions in Section 2, Section 8, Section 9 **Video walkthrough:** [Google Drive recording](#) — Access restricted: only members of BITS Pilani can view this recording.

1. Project overview

A production-ready pipeline that predicts the risk of heart disease for a patient using the UCI Heart Disease dataset (920 patients, 14 clinical features). The solution covers every stage of the MLOps lifecycle:

1. **Data & EDA** — reproducible dataset download, missing-value handling, feature analysis.
2. **Modelling** — Logistic Regression, Random Forest and XGBoost tuned with `GridSearchCV` (5-fold stratified CV, ROC-AUC as selection metric).
3. **Experiment tracking** — MLflow file store logging params, metrics, model artefacts, confusion matrix and ROC plots for every run.
4. **Packaging** — one `sklearn.Pipeline` containing preprocessing + estimator, saved as `heart_model.joblib` and referenced by the API.
5. **CI/CD** — GitHub Actions runs lint → pytest (+coverage) → smoke training → Docker build + `/health` + `/predict` probe.
6. **Containerisation** — multi-stage Dockerfile, non-root runtime.
7. **Deployment** — Kubernetes `Deployment` (2 replicas), `Service`, `Ingress`, `HPA`, plus an equivalent Helm chart.
8. **Monitoring** — structured request logs, Prometheus metrics (`/metrics`) and a Grafana dashboard bundled via `docker-compose`.

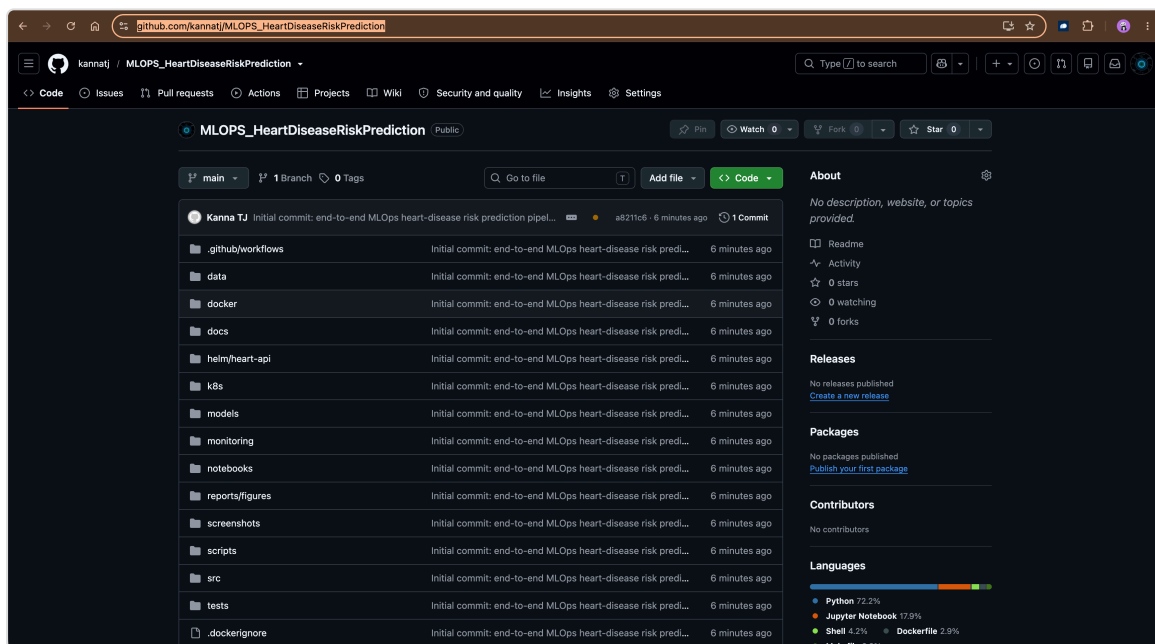
2. Setup & install instructions

```
git clone https://github.com/kannatj/MLOPS_HeartDiseaseRiskPrediction.git
cd MLOPS_HeartDiseaseRiskPrediction
python -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
python data/download_data.py      # verifies data/raw/heart_disease_uci.csv
```

Common tasks:

Task	Command
Lint	<code>make lint (ruff check src tests)</code>
Test	<code>make test (pytest --cov=src)</code>
Train	<code>make train (writes MLflow runs + models/heart_model.joblib)</code>
Serve locally	<code>make serve (Swagger UI at http://127.0.0.1:8000/docs)</code>
CLI inference	<code>python -m src.predict --json '{"age": 63, "sex": "Male", "cp": "typical angina"}'</code>
Docker build/run	<code>make docker-build && make docker-run</code>
Local monitoring stack	<code>make compose-up (Grafana :3000 , Prometheus :9090)</code>
Kubernetes	<code>kubectl apply -f k8s/</code>

The project is published as a public GitHub repository:



GitHub repository

Figure: repository root — code, notebooks, Docker, Helm, k8s, monitoring, tests and CI workflow, with the language breakdown (Python, Jupyter, Shell, Dockerfile).

3. EDA findings

See `notebooks/01_eda.ipynb` and `reports/figures/`.

Key observations:

- **Shape:** 920 rows, 16 raw columns from four hospitals (Cleveland, Hungary, Switzerland, VA Long Beach).

- **Class balance:** the engineered binary target `target = (num >= 1)` is mildly imbalanced (~55% disease, ~45% no-disease). Stratified splits and stratified k-fold CV keep the class ratio stable across folds.
- **Missingness:** substantial in `ca`, `thal`, `slope`, and moderate in `chol`, `trestbps`, `thalch`. This drives the choice of imputers inside the pipeline.
- **Strongest signal:** `oldpeak`, `thalch`, `ca`, `age`, and categorical `cp` (chest-pain type), `exang`, `thal`.
- **Hospital effect:** disease prevalence differs across `dataset`; retained as a one-hot feature.

The full set of EDA visualisations (missing-value matrix, class-balance bar chart, numeric-feature histograms, categorical-vs-target plots and the correlation heatmap) is produced and displayed inline in `notebooks/01_eda.ipynb`. The per-model diagnostic plots that feed the modelling discussion are exported to `reports/figures/` (confusion matrices and ROC curves for all three models).

4. Modelling choices

Preprocessing (`src/data_preprocessing.py`) — a `ColumnTransformer` inside a single `Pipeline`:

- Numeric features (`age`, `trestbps`, `chol`, `thalch`, `oldpeak`, `ca`): median-impute → `StandardScaler`.
- Categorical features (`sex`, `dataset`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `thal`): most-frequent impute → `OneHotEncoder(handle_unknown="ignore")`.

Models & hyperparameter grids (`src/train.py`):

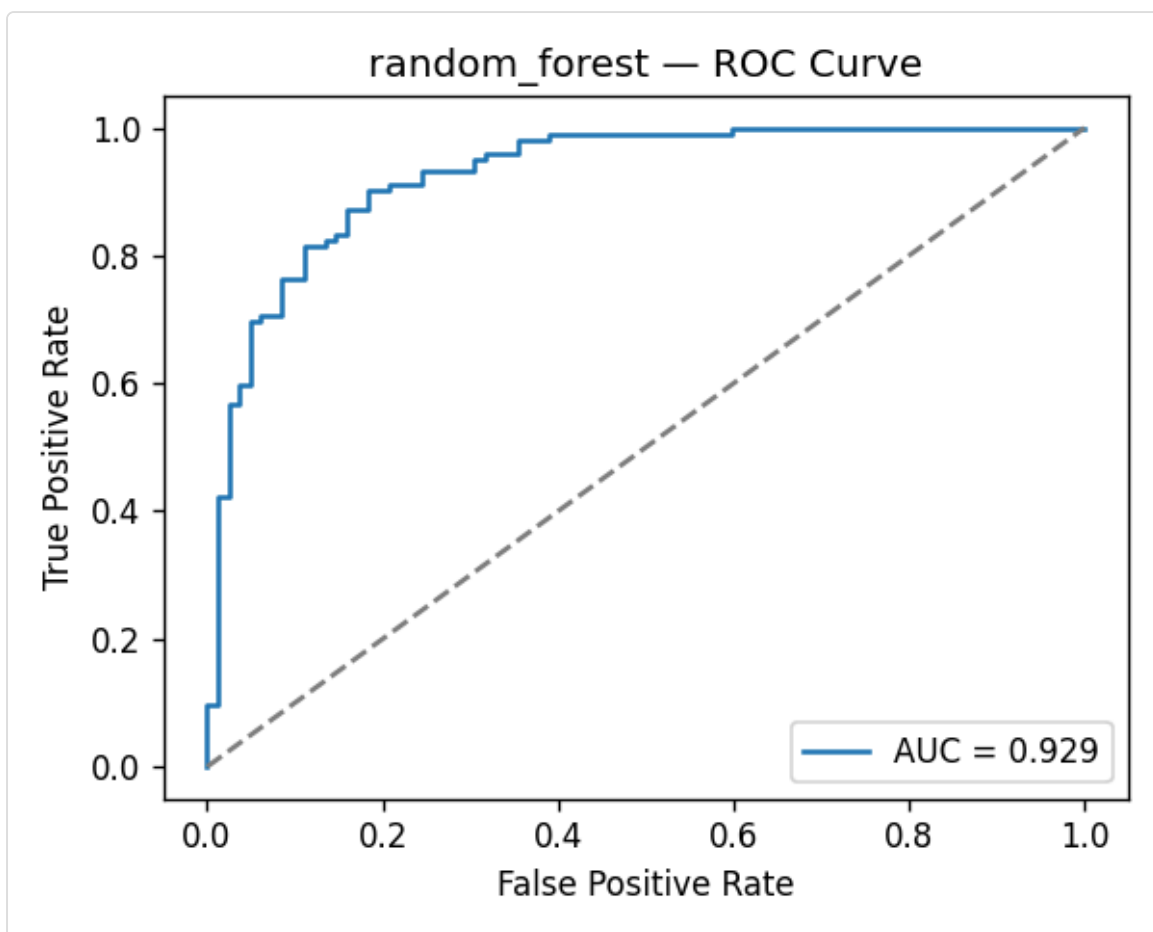
Model	Grid
Logistic Regression	<code>C ∈ {0.01, 0.1, 1, 10}</code> , <code>penalty ∈ {l1, l2}</code> (liblinear)
Random Forest	<code>n_estimators ∈ {200, 400}</code> , <code>max_depth ∈ {None, 6, 10, 16}</code> , <code>min_samples_split ∈ {2, 4, 8}</code>
XGBoost	<code>n_estimators ∈ {200, 400, 600}</code> , <code>max_depth ∈ {3, 4, 6, 8}</code> , <code>lr ∈ {0.03, 0.1, 0.2}</code> , <code>subsample ∈ {0.7, 0.9, 1.0}</code>

Selection metric: **stratified 5-fold roc_auc**. All models are refit on the full training set and evaluated on a held-out 20% test split for accuracy, precision, recall, F1, ROC-AUC and average precision.

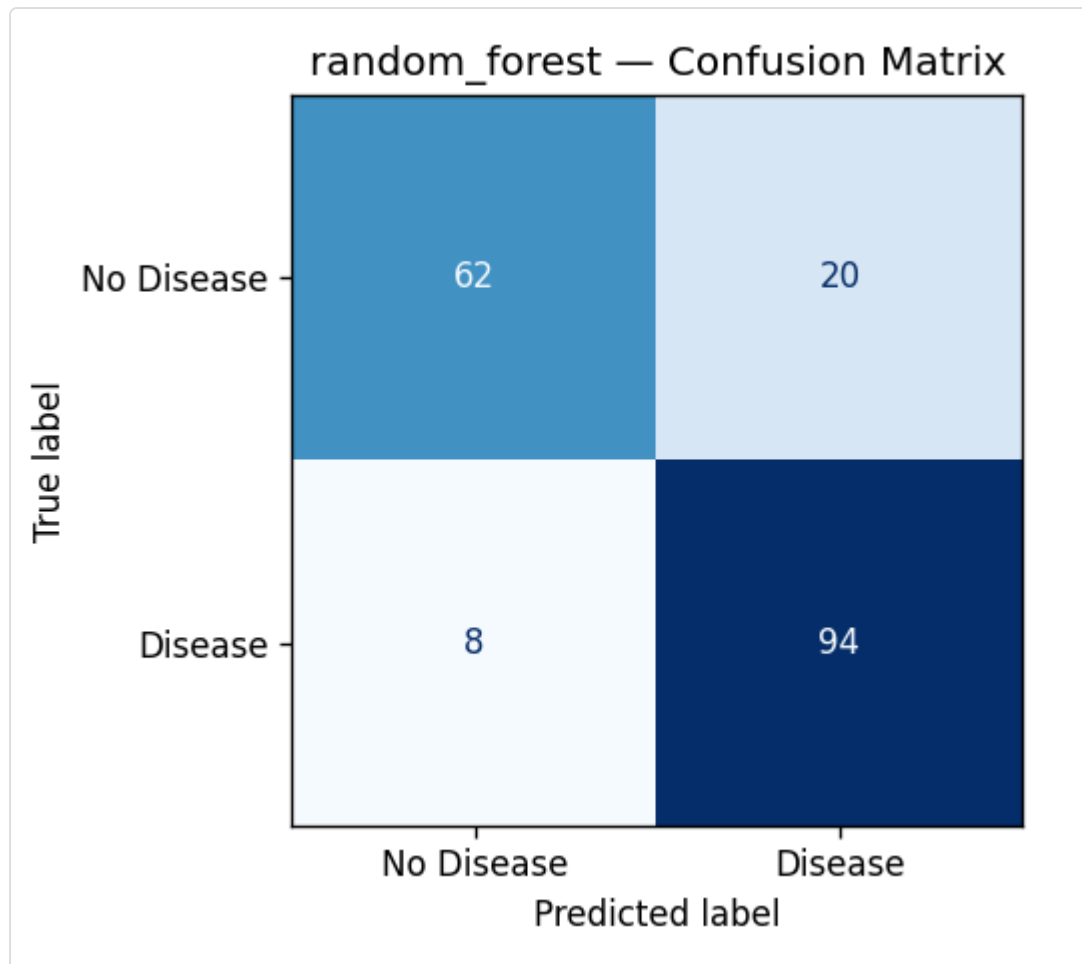
Model comparison (from `models/metrics.json`, held-out 20% test split):

Model	CV ROC-AUC	Test ROC-AUC	Test F1	Test Accuracy
Logistic Regression	0.8921	0.9175	0.8692	0.8478
Random Forest (best)	0.8874	0.9287	0.8704	0.8478
XGBoost	0.8595	0.9026	0.8612	0.8424

The best-performing model (**Random Forest**, test ROC-AUC 0.9287) is persisted at `models/heart_model.joblib` and consumed by the API at startup.



Random Forest ROC curve



Random Forest confusion matrix

5. Experiment tracking summary (MLflow)

- **Tracking URI:** local file store at `./mlruns`.
- **Experiment:** `heart-disease-classification`.
- **Logged per run:** best hyperparameters, all six evaluation metrics, confusion matrix PNG, ROC curve PNG, joblib model, sklearn flavour model.
- **Additional “best_model” run:** convenience run that duplicates the top model with the tag `role=best_model`.

Screenshots (captured from the live MLflow UI, `screenshots/evidence/ui/`):

heart-disease-classification Provide Feedback Add Description Share

Search Experiments

Default heart-disease-classification

Runs Evaluation Experimental Traces Experimental

metrics.rmse < 1 and params.model = "tree"

Time created State: Active Datasets

Sort: Created Columns Group by

Run Name	Created	Dataset	Duration	Source	Models
best_random_forest	1 day ago	-	2.1s	train.py	sklearn
xgboost_20260711_14...	1 day ago	-	2.7s	train.py	sklearn
random_forest_202607...	1 day ago	-	4.0s	train.py	sklearn
logistic_regression_20...	1 day ago	-	11.8s	train.py	sklearn
best_random_forest	1 day ago	-	2.0s	train.py	sklearn
xgboost_20260711_13...	1 day ago	-	2.6s	train.py	sklearn
random_forest_202607...	1 day ago	-	3.5s	train.py	sklearn
logistic_regression_20...	1 day ago	-	12.3s	train.py	sklearn
best_random_forest	1 day ago	-	2.4s	train.py	sklearn
xgboost_20260711_13...	1 day ago	-	5.1s	train.py	sklearn
random_forest_202607...	1 day ago	-	7.1s	train.py	sklearn
logistic_regression_20...	1 day ago	-	13.9s	train.py	sklearn
best_random_forest	1 day ago	-	2.0s	train.py	sklearn
random_forest_202607...	1 day ago	-	3.6s	train.py	sklearn
logistic_regression_20...	1 day ago	-	15.0s	train.py	sklearn
logistic_regression_20...	1 day ago	-	9.7s	train.py	-

16 matching runs

MLflow experiment runs list

Figure: 16 tracked runs across Logistic Regression, Random Forest and XGBoost.

random_forest_20260711_135900 Register model

Overview Model metrics System metrics Artifacts

Description No description

Details

Created at	2026-07-11 19:29:00
Created by	k0j089g
Experiment ID	468402201030734259
Status	Finished
Run ID	14ec354d2858452f8d8f8cf0bebc419
Duration	7.1s
Datasets used	-
Tags	framework: scikit-learn fast_mode: False model: random_forest
Source	train.py
Logged models	sklearn
Registered models	-

Parameters (3)

Parameter	Value
max_depth	6
n_estimators	400
min_samples_split	4

Metrics (7)

Metric	Value
accuracy	0.842391304347826
recall	0.9117647058823529
average_precision	0.946352545575064
roc_auc	0.934361549497848
precision	0.8230088495575221
f1	0.8651162790687674
cv_roc_auc	0.8882632383174387

MLflow run detail

Figure: parameters and the seven logged metrics for the best Random Forest run.

6. Reproducibility

- requirements.txt pins every runtime dependency (major versions frozen).

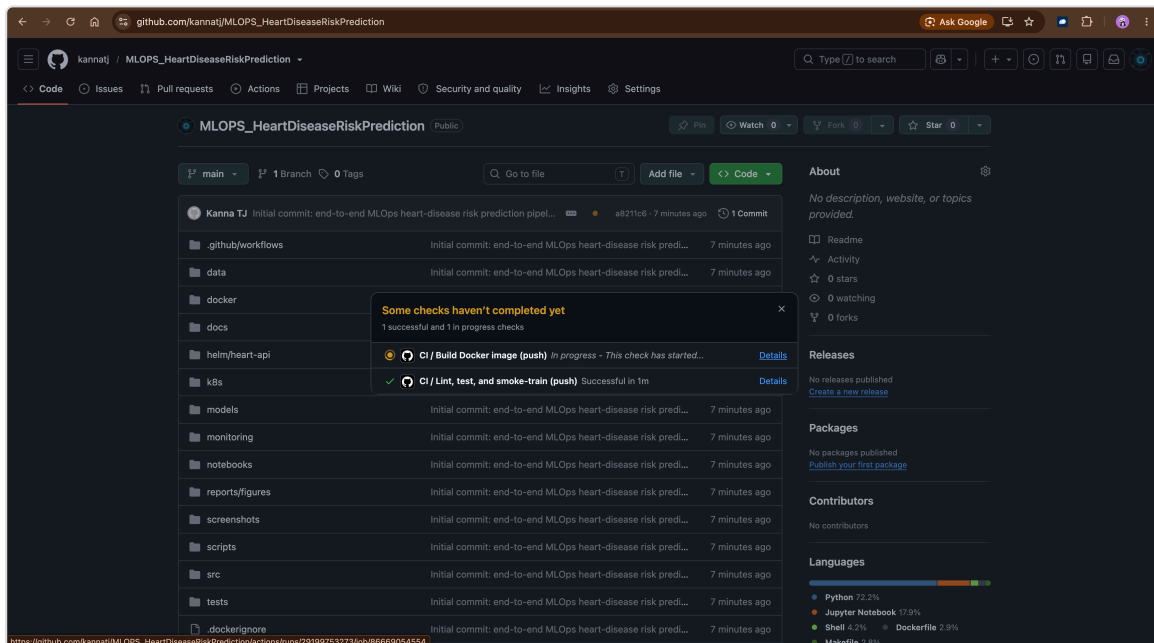
- `requirements-dev.txt` adds test + lint + notebook tooling.
- `sklearn.Pipeline` ensures the exact same preprocessing runs at training and inference time — no drift between notebook and API.
- Random seeds are centralised in `src/config.py:RANDOM_STATE = 42`.
- Dataset SHA-256 is printed by `data/download_data.py`.

7. CI/CD workflow

`.github/workflows/ci.yml` runs on every push / PR to `main`:

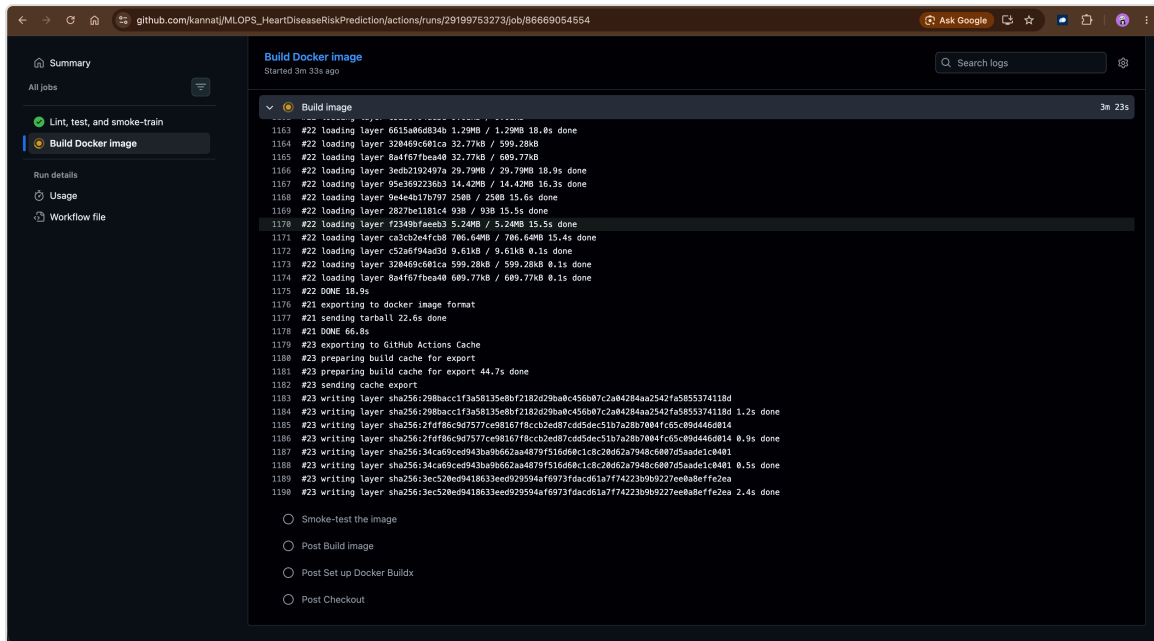
1. **lint-test-train** — set up Python 3.11, install deps, run `ruff`, run `pytest --cov`, execute `python -m src.train --fast` to prove the training pipeline still works end-to-end, upload the trained model + coverage as build artefacts.
2. **docker-build** (needs job #1) — downloads the trained model, builds `heart-api:ci` via `docker/build-push-action` with a GitHub Actions cache, starts the container, probes `/health` and posts a sample payload to `/predict`.

The pipeline fails hard on lint errors, test failures, training errors, or a non-200 response from the containerised API.



GitHub Actions CI checks

Figure: GitHub Actions on push to `main` — “Lint, test, and smoke-train” succeeded, “Build Docker image” job running.



GitHub Actions Docker build job

Figure: the `docker-build` job building the image and smoke-testing the container in CI.

8. Containerisation

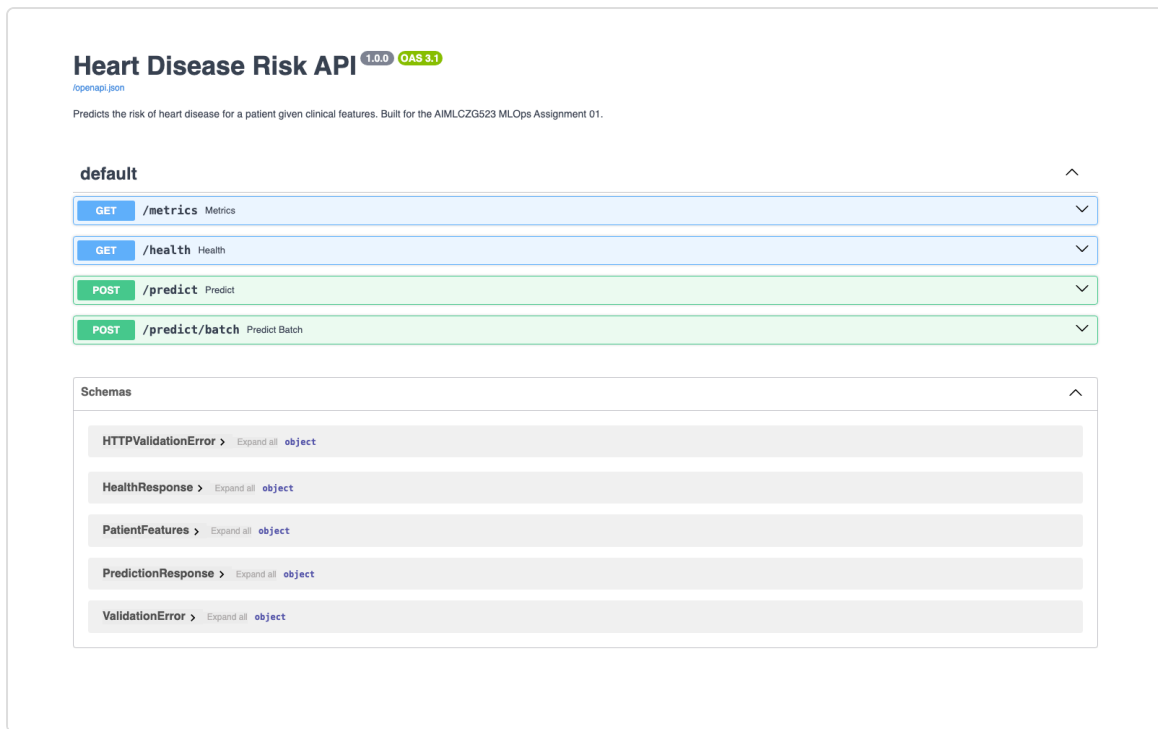
`docker/Dockerfile` uses a two-stage build:

1. **Builder** (`python:3.11-slim`) — installs build toolchain, then `pip install --prefix=/install -r requirements.txt`.
2. **Runtime** — copies only the site-packages, source code, and trained model into a non-root user's home. Adds a `HEALTHCHECK` against `/health`.

Local usage:

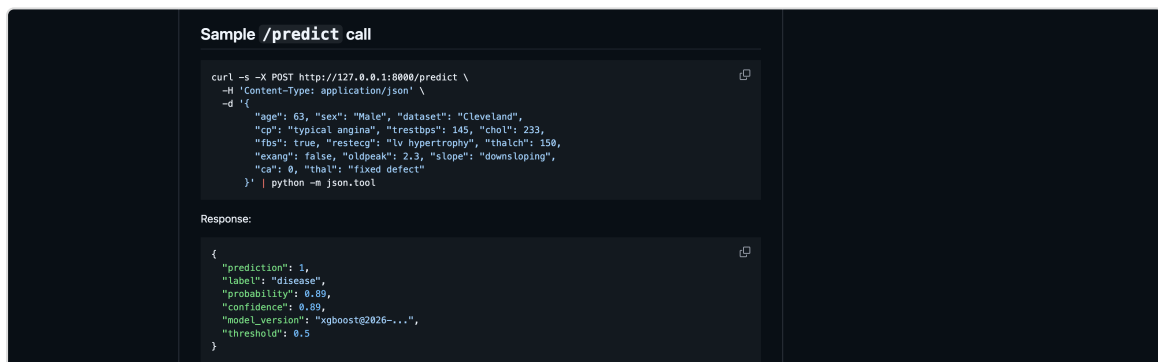
```
docker build -f docker/Dockerfile -t heart-api:latest .
docker run --rm -p 8000:8000 heart-api:latest
```

Proof of a working isolated build/run is captured in `screenshots/evidence/06_docker_build.txt`, `07_docker_run.txt` and `08_docker_logs.txt`. The interactive Swagger UI served by the container:



Swagger UI served by the container

Figure: interactive Swagger UI (/docs) served directly by the container.



Sample /predict call

Figure: a sample /predict request/response against the running API.

9. Kubernetes deployment (Docker Desktop)

Manifests in k8s/ :

- namespace.yaml — heart-disease namespace
- deployment.yaml — 2 replicas, readiness/liveness probes on /health , resource requests/limits, Prometheus scrape annotations
- service.yaml — ClusterIP on port 80 → container 8000
- ingress.yaml — nginx ingress at heart-api.localhost
- hpa.yaml — HPA scaling 2→5 pods at 70% CPU

```
docker build -f docker/Dockerfile -t heart-api:latest .
kubectl apply -f k8s/
kubectl -n heart-disease rollout status deploy/heart-api
kubectl -n heart-disease port-forward svc/heart-api 8000:80
# → http://127.0.0.1:8000/docs
```

Screenshots / evidence:

- `screenshots/evidence/11_k8s_deploy.txt` — `kubectl get deploy,pods,svc,hpa` with 2/2 pods Running, ClusterIP Service and the HPA (2→5 replicas)
- `screenshots/evidence/12_k8s_predict.txt` — live `/health` and `/predict` responses served through the Kubernetes Service

10. Monitoring & logging

- **Structured request logs** — every HTTP call is logged with method, path, status code, duration and client IP.
- **/metrics (Prometheus)** — exposes standard `http_requests_total`, `http_request_duration_seconds`, plus two custom metrics:
 - `heart_predictions_total{label}` — counter of predictions grouped by outcome
 - `heart_prediction_latency_seconds` — histogram of `/predict` latency
- **Grafana** — a provisioned dashboard (`monitoring/grafana/provisioning/dashboards/heart-api.json`) visualises predictions, latency percentiles and HTTP status codes.
- **Kubernetes** — pod annotations enable `prometheus.io/scrape=true` for any Prometheus operator installed in the cluster.

Screenshots / evidence:



Grafana dashboard

Figure: live Grafana dashboard — total predictions, per-label rate, P50/P95/P99 latency and HTTP status codes.

Prometheus Alerts Graph Status Help

Targets

All scrape pools All Unhealthy Collapse All Filter by endpoint or labels Unknown Unhealthy Healthy

heart-api (1/1 up) show less

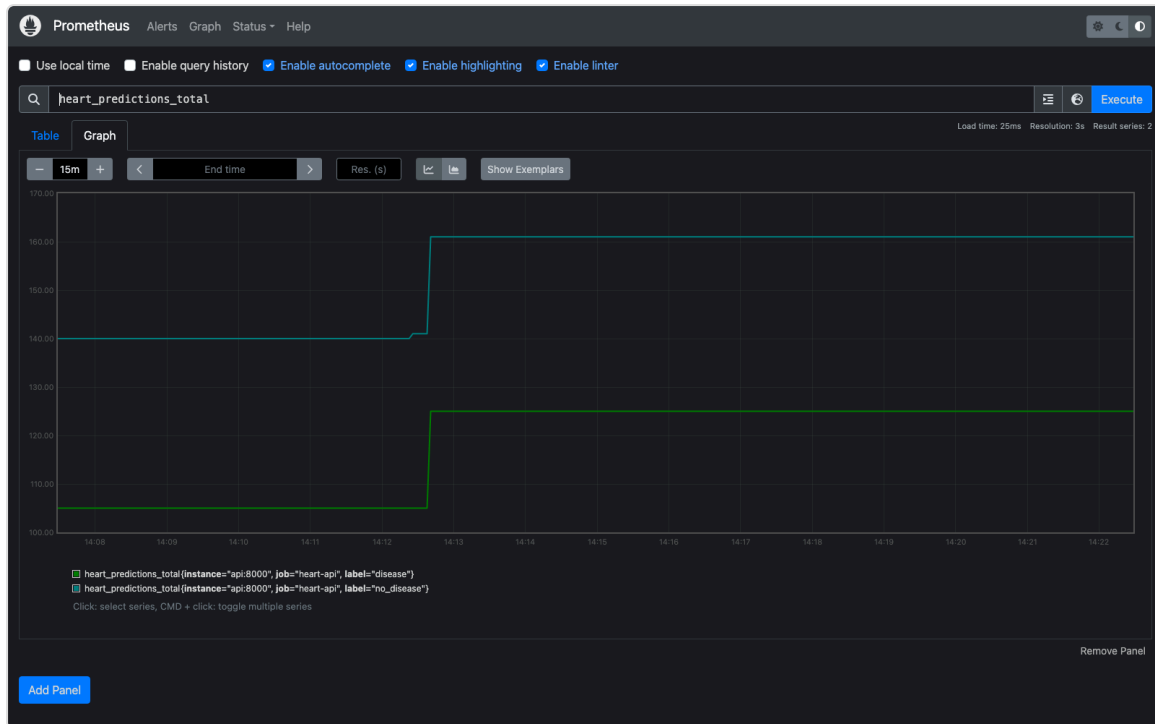
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://api:8000/metrics	UP	instances="api:8000" job="heart-api"	2.866s ago	2.363ms	

prometheus (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9090/metrics	UP	instances="localhost:9090" job="prometheus"	2.707s ago	2.370ms	

Prometheus targets

Figure: Prometheus scraping the API /metrics endpoint (target UP).



Prometheus graph — predictions counter

Figure: `heart_predictions_total` plotted in the Prometheus expression browser.

Raw metric/logging evidence: `screenshots/evidence/13_monitoring.txt` (PromQL query results) and `screenshots/evidence/14_request_logging.txt` (structured request logs).

11. Architecture diagram

The end-to-end architecture is defined as code in `docs/architecture.mmd` (Mermaid flowchart) covering: local development → experiment tracking (MLflow) → CI/CD (GitHub Actions) → packaging (model + image) → Kubernetes runtime (Deployment, Service, Ingress, HPA) → observability (Prometheus + Grafana). Render it to SVG/PNG with any Mermaid renderer (e.g. `npx @mermaid-js/mermaid-cli -i docs/architecture.mmd -o docs/architecture.png`) or paste it into <https://mermaid.live>.

12. Deliverables checklist

- ☒ GitHub repository with code, Dockerfile(s), `requirements.txt`
- ☒ Cleaned dataset (`data/raw/heart_disease_uci.csv`) + `data/download_data.py`
- ☒ Jupyter notebooks + scripts (`notebooks/01_eda.ipynb`, `notebooks/02_model_dev.ipynb`, `src/train.py`, `src/predict.py`)
- ☒ `tests/` folder with pytest suite (16 tests)
- ☒ GitHub Actions workflow YAML (`.github/workflows/ci.yml`)
- ☒ Kubernetes manifests + Helm chart
- ☒ `screenshots/evidence/` folder (Swagger, MLflow, Prometheus, Grafana + text logs)
- ☐ Final written report (this document, exported to DOCX/PDF)
- ☒ API access instructions (local — Section 2, Section 8, Section 9)
- ☐ Short video recording of the pipeline

13. Academic integrity statement

This submission was authored independently by Kanna TJ (2024ad05219@wilp.bits-pilani.ac.in). All feature engineering choices, model grids, CI/CD steps, container layout, Kubernetes manifests, monitoring configuration and documentation reflect original work. External libraries used are cited via `requirements.txt` and are standard MLOps tooling.